

(Lack of) DevOps Case Study

Dev the developer, Tess the tester, and Optimus from Operations were all working on the same project.

Dev wasn't even sure where the latest version of the code lived. Another developer emailed him a zip file of source code and told him to use that. Even then, Dev couldn't get the system running on his machine. Every time he tried, a new missing dependency appeared, something else he had to track down, install, and configure.

Meanwhile, other developers had finished their features and were keen to release. "You can't release yours until mine is ready too," became a common phrase.

So they moved on to other work while waiting. But that work only got halfway done before the cycle repeated. Nothing was ever truly finished, just paused and restarted.

Tess carried out testing in the staging environment and quickly found multiple issues. "But it works on my machine," the developers insisted. "If I can't reproduce it, I can't fix it."

When bugs were eventually fixed, Tess often had to wait weeks before she could test them again.

"Every time you want a new build, we have to package and deploy it manually," the developers complained "That takes a full day away from development."

Eventually, the release date set by Marketing arrived.

To deploy the new version, Optimus had to shut down the live system. "No one will be using it at this time," he assumed, as he brought down the servers and began copying files over manually. He noticed configuration files with different values but copied them anyway.

After half an hour of frantic work, the system was back online.

Dev eagerly opened the live site and clicked on his new feature. His excitement quickly turned to confusion, then frustration. He was looking at the old, buggy version. Where had his changes gone?

Across the business, things were no better. Features that had previously worked were now broken.

Optimus was asked to roll back the release. "Sorry," he said, "I don't have a copy of the old version or its configuration."

Until a new version could be created, the broken system would remain live.

Complaints about performance soon followed. Tess pointed out that everything had seemed fine in testing. Optimus explained that the live system included additional firewalls and antivirus tools that weren't present in staging.

Then things got worse. A national news story mentioned the site, causing a sudden surge in traffic. Visitors were greeted with: **Error 500 – Server Busy**

"We've added more memory and another network card," said Optimus, but the application only runs on a single server. There's nothing else we can do.

Think about and discuss

- What went wrong on this project?
- What could have been done differently?
- Have you seen similar "horror stories" in real projects?